Supplemental Material for Mutualism can constrain host diversity and evolution

2026-03-19

Contents

1 Introduction **5**

1.1 About our supplemental material 5

1.2 Contributing authors 5

2 Data availability **7**

2.1 Source code 7

2.2 Experiment results 7

3 Compilation instructions **9**

3.1 Instructions 9

4 Virtual hardware for digital organisms **11**

4.1 Virtual hardware components 11

4.2 Instruction set 11

5 Equal function rewards **15**

5.1 Dependencies and setup 15

5.2 Load and preprocess data 17

5.3 Extinctions 18

5.4 Host function performance 20

5.5 Symbiont function performance 26

5.6 Host phenotypic diversity 32

5.7 Symbiont phenotypic diversity 38

5.8 Trait matches over time 43

5.9 Evolutionary engagement 44
5.10 Generations of evolution elapsed 46
5.11 Symbiont transmission rates 51
5.12 Population size 56
5.13 Manuscript figures 59

Chapter 1

Introduction

This is the supplemental material our 2026 Artificial Life Conference submission, "Mutualism can constrain host diversity and evolution". This is not intended as a stand-alone document, but as a companion to our main manuscript.

1.1 About our supplemental material

Our supplemental material is hosted using GitHub pages. We compiled our data analyses and supplemental documentation into this web-accessible book using bookdown.

The source code and configuration files for this supplemental material can be found in this GitHub repository.

Our supplemental material includes the following:

- Data availability (Section 2)
- Local compilation instructions (Section 3)

TODO

1.2 Contributing authors

- Delaney Kelley
- Owen Hillary
- Elias Hillman-Emelianoff
- Ahmad Khalili
- Emma Zhang

- Kiara M. Johnson
- Emily Dolson
- Anya E. Vostinar
- Alexander Lalejini

Chapter 2

Data availability

2.1 Source code

The source code for his work is publicly accessible on GitHub: https://github.com/amlalejini/. This repository has also been archived on Zenodo: TODO

2.2 Experiment results

Data generated from our experiments used in analyses are available online, archived in an OSF repository: https://osf.io/j3ezd/

Chapter 3

Compilation instructions

Instructions for compiling and running the software used in this study on your local machine. All experiments were run on Mac or Linux-based operating systems.

You will need a C++ compiler that supports C++20. We used g++14 for all local compilations.

You will also need Python to run graph generation and analysis. Python dependencies are listed in the `requirements.txt` at the root of this repository.

Statistical analyses and data visualizations were conducted using R.

Experiments were conducted using the Symbulation software platform. The specific version of Symbulation used for these experiments can be found here: https://github.com/amlalejini/SymbulationEmp/tree/sgp-mode-refactor (commit hash: `476bd8a4c0e6390942ede46ee9cd1bbe5ab8ec70`). This supplemental repository has the correct Symbulation version pinned as a git submodule (in `third-party/`).

3.1 Instructions

First, clone the `2026-alife-mutualism-constrains-evolution` repository (https://github.com/amlalejini/2026-alife-mutualism-constrains-evolution) to your local machine. Then, initialize and update the git submodules inside the repository. From inside the repository, run:

```
git submodule update --init --recursive
```

This will download the correct version of Symbulation along with all of its necessary dependencies.

To compile Symbulation, navigate to the `third-party/SymbulationEmp` directory, and run `make sgp-mode`. The compiled executable will be placed inside the `third-party/SymbulationEmp` directory.

The Symbulation configuration files used for each of experiments can be found in the `experiments/` directory (within the `hpc/config/` subdirectory for any given experiment).

Chapter 4

Virtual hardware for digital organisms

4.1 Virtual hardware components

Each digital organism comprises a linear program (its genome) and a set of virtual hardware components for executing its genome. Genomes use the SignalGP-lite representation (https://github.com/mmore500/signalgp-lite).

In this work, the virtual hardware contains: - 8 registers: these are used as "working memory", used as source operands and destination operands for instructions - 2 memory stacks: serve as additional memory for programs. Programs can push values from registers to a stack or pop values from a stack into a register. Each stack has a maximum size of 16 values.

4.2 Instruction set

SignalGP instruction set used in digital organism genomes in this work.

Abbreviations:

- EOP: End of program
- Reg: local register
  - Reg[0] indicates the value at the register specified by an instruction's first *argument*, Reg[1] indicates the value at the register specified by an instruction's second argument, and Reg[2] indicates the value at the register specified by the instruction's third argument.
  - Reg[0], Reg[1], *etc*: Register 0, Register 1, *etc.*

- Arg: Instruction argument
  - Arg[i] indicates the i'th instruction argument (an integer encoded in the genome)
  - E.g., Arg[0] is an instruction's first argument

Instructions that would produce undefined behavior (e.g., division by zero) are treated as no operations.

Instruction	Arguments Used	Description
`Nop`	0	No operation
`Increment`	1	Reg[0] = Reg[0] + 1
`Decrement`	1	Reg[0] = Reg[0] - 1
`ShiftLeft`	1	Reg[0] = Reg[0] « 1
`ShiftRight`	1	Reg[0] = Reg[0] » 1
`Subtract`	3	Reg[0] = Reg[1] - Reg[2]
`Nand`	3	Reg[0] = ~(Reg[1] & Reg[2])
`Push`	1	Push Reg[0] to active memory stack
`Pop`	1	Pop top of active memory stack. Reg[0] = popped value
`SwapStack`	0	Change active stack.
`Swap`	2	Swap values between Reg[0] and Reg[1]
`Reproduce`	0	Triggers organism reproduction attempt if organism has sufficient resources.
`IO`	1	Output value in Reg[0], and set Reg[0] = new input value.
`JumpIfNEq`	2	if (Reg[0] != Reg[1]) Jump to Anchor specified by instruction tag (tag matching determines target Anchor).

Instruction	Arguments Used	Description
`JumpIfLess`	2	if (Reg[0] < Reg[1]) Jump to Anchor specified by instruction tag (tag matching determines target Anchor).
`JumpIfEq`	2	if (Reg[0] == Reg[1]) Jump to Anchor specified by instruction tag (tag matching determines target Anchor).
`Anchor`	0	Tag-based code label that can be jumped to with a jump instruction.

Chapter 5

Equal function rewards

All logic functions yield equal resources when performed by digital organisms.
No "abiotic" selective pressure to evolve more complex functions.

5.1 Dependencies and setup

Load R library dependencies.

```
library(tidyverse)
library(cowplot)
library(khroma)
library(rstatix)
library(knitr)
library(kableExtra)
```

Define relevant file paths.

```
# Check if Rmd is being compiled using bookdown
bookdown <- exists("bookdown_build")

# Build relative path from R's working directory to what is actually the working
# directory for this experiment.
experiment_slug <- "2025-11-24-health-flat-rewards"
exp_directory <- paste(
  "experiments",
  experiment_slug,
  "analysis",
  sep = "/"
```

```
)

# Get path to the supplemental material analysis directory
supp_directory <- paste(
  "experiments",
  "manuscript-analyses",
  sep = "/"
)

# Adjust experiment directory if being knitted for bookdown build
if (bookdown) {
  exp_directory <- paste0(
    bookdown_wd_prefix,
    exp_directory
  )
  supp_directory <- paste0(
    bookdown_wd_prefix,
    supp_directory
  )
}

supp_file_prefix <- "flat-rewards"
```

Define default plot theme, plot output directory.

```
# Use the cowplot theme as our default for ggplots
theme_set(theme_cowplot())
# Create directory to save plots
plot_dir <- paste(
  supp_directory,
  "plots",
  supp_file_prefix,
  sep = "/"
)
dir.create(
  plot_dir,
  showWarnings = FALSE
)

# Use vibrant theme for plot color palettes
vibrant_colors <- color("vibrant")(7)
color_pal <- c(
  vibrant_colors[["blue"]],
  vibrant_colors[["orange"]],
  vibrant_colors[["magenta"]]
```

```
)
```

5.2 Load and preprocess data

```
tasks <- c("NOT", "NAND", "OR_NOT", "AND", "OR", "AND_NOT", "NOR", "XOR", "EQU")
max_pop_size <- 100 * 100
threshold <- 0.025 * max_pop_size
```

Load summary data.

```
summary_data_path <- paste(
  exp_directory,
  "data",
  "summary.csv",
  sep = "/"
)
summary_data <- read_csv(summary_data_path)
# Do any universal preprocessing
summary_data <- summary_data %>%
  mutate(
    SEED = as.factor(SEED),
    task_credit = paste0(
      "H", HOST_ONLY_FIRST_TASK_CREDIT,
      "S", SYM_ONLY_FIRST_TASK_CREDIT
    ),
    sym_condition = case_when(
      START_MOI == "0" ~ "no-syms",
      START_MOI == "1" & HEALTH_TYPE == "parasite" ~ "parasite",
      START_MOI == "1" & HEALTH_TYPE == "mutualist" ~ "mutualist",
      .default = "unknown" # precaution against unexpected parameterizations
    ),
    interaction_multiplier = as.factor(PARASITE_CYCLE_STEAL_MULTIPLIER)
  ) %>%
  mutate(
    task_credit = as.factor(task_credit),
    sym_condition = factor(
      sym_condition,
      levels = c("no-syms", "parasite", "mutualist")
    )
  ) %>%
  # Filter to just IM=4 data => that's our baseline in the paper.
  filter(interaction_multiplier == "4")
```

Load time series data.

```
time_data_path <- paste(
  exp_directory,
  "data",
  "time_series.csv",
  sep = "/"
)
time_data <- read_csv(time_data_path)

# Do any universal preprocessing
time_data <- time_data %>%
  mutate(
    SEED = as.factor(SEED),
    sym_condition = case_when(
      START_MOI == "0" ~ "no-syms",
      START_MOI == "1" & HEALTH_TYPE == "parasite" ~ "parasite",
      START_MOI == "1" & HEALTH_TYPE == "mutualist" ~ "mutualist",
      .default = "unknown" # precaution against unexpected parameterizations
    ),
    interaction_multiplier = as.factor(PARASITE_CYCLE_STEAL_MULTIPLIER)
  ) %>%
  mutate(
    sym_condition = factor(
      sym_condition,
      levels = c("no-syms", "parasite", "mutualist")
    )
  ) %>%
  # Filter to just IM=4 data => that's our baseline in the paper.
  filter(interaction_multiplier == "4")
```

5.3 Extinctions

Did host or symbiont populations go extinct in any runs?

```
sym_min_pop_sizes <- time_data %>%
  group_by(SEED) %>%
  slice_min(
    OrgCounts_hosted_sym_count,
    n = 1
  ) %>%
  slice_min(
    update,
    n = 1
```

```
  )

reps_with_extinct_syms <- sym_min_pop_sizes %>%
  group_by(
    sym_condition
  ) %>%
  summarize(
    reps_with_extinct_syms = sum(OrgCounts_hosted_sym_count == 0),
    total_reps = n()
  )

host_min_pop_sizes <- time_data %>%
  group_by(SEED) %>%
  slice_min(
    OrgCounts_host_count,
    n = 1
  ) %>%
  slice_min(
    update,
    n = 1
  )

reps_with_extinct_hosts <- host_min_pop_sizes %>%
  group_by(
    sym_condition
  ) %>%
  summarize(
    reps_with_extinct_hosts = sum(OrgCounts_host_count == 0),
    total_reps = n()
  )

extinctions <- full_join(
  reps_with_extinct_syms,
  reps_with_extinct_hosts
)
extinctions <- extinctions %>%
  arrange(
    desc(reps_with_extinct_hosts), desc(reps_with_extinct_syms)
  )
extinction_table <- kable(extinctions) %>%
  kable_styling(latex_options = "striped")
save_kable(
  extinction_table,
  paste(
    plot_dir,
```

```
    "extinctions.pdf",
    sep = "/"
  )
)

extinctions
```

```
## # A tibble: 3 x 4
##   sym_condition reps_with_extinct_syms total_reps reps_with_extinct_hosts
##   <fct>                          <int>      <int>                   <int>
## 1 no-syms                           50         50                       0
## 2 parasite                           0         50                       0
## 3 mutualist                          0         50                       0
```

5.4 Host function performance

First, quantify which functions have been performed over the course of an entire run.

```
# Pivot task data long, selecting only relevant columns
host_over_time_parent_task_data <- time_data %>%
  select(
    SEED,
    sym_condition,
    interaction_multiplier,
    update,
    CurUpdate_AND_NOT_in_host_parent_org_counts,
    CurUpdate_AND_in_host_parent_org_counts,
    CurUpdate_EQU_in_host_parent_org_counts,
    CurUpdate_NAND_in_host_parent_org_counts,
    CurUpdate_NOR_in_host_parent_org_counts,
    CurUpdate_NOT_in_host_parent_org_counts,
    CurUpdate_OR_NOT_in_host_parent_org_counts,
    CurUpdate_OR_in_host_parent_org_counts,
    CurUpdate_XOR_in_host_parent_org_counts
  ) %>%
  pivot_longer(
    c(
      CurUpdate_AND_NOT_in_host_parent_org_counts,
      CurUpdate_AND_in_host_parent_org_counts,
      CurUpdate_EQU_in_host_parent_org_counts,
      CurUpdate_NAND_in_host_parent_org_counts,
      CurUpdate_NOR_in_host_parent_org_counts,
```

```
      CurUpdate_NOT_in_host_parent_org_counts,
      CurUpdate_OR_NOT_in_host_parent_org_counts,
      CurUpdate_OR_in_host_parent_org_counts,
      CurUpdate_XOR_in_host_parent_org_counts
    ),
    names_to = "task",
    values_to = "task_in_parent_count"
  ) %>%
  mutate(
    task = str_replace(task, "CurUpdate_", "")
  ) %>%
  mutate(
    task = str_replace(task, "_in_host_parent_org_counts", "")
  ) %>%
  mutate(
    task_completed_by_parent = task_in_parent_count >= threshold,
    task = factor(
      task,
      levels = tasks,
      labels = tasks
    )
  ) %>%
  mutate(
    task_complexity = case_when(
      task == "NOT" ~ 1,
      task == "NAND" ~ 1,
      task == "OR_NOT" ~ 2,
      task == "AND" ~ 2,
      task == "OR" ~ 3,
      task == "AND_NOT" ~ 3,
      task == "NOR" ~ 4,
      task == "XOR" ~ 4,
      task == "EQU" ~ 5,
      .default = -1
    )
  )

host_parent_tasks_all_time_by_task <- host_over_time_parent_task_data %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier,
    task
  ) %>%
  summarize(
```

```
    n = n(),
    parent_task_completed_ever = any(task_completed_by_parent)
  ) %>%
  ungroup() %>%
  group_by(
    sym_condition,
    interaction_multiplier,
    task
  ) %>%
  summarize(
    n = n(),
    parent_tasks_completed_ever = sum(parent_task_completed_ever == TRUE)
  )
```

Next, summarize function complexity level.

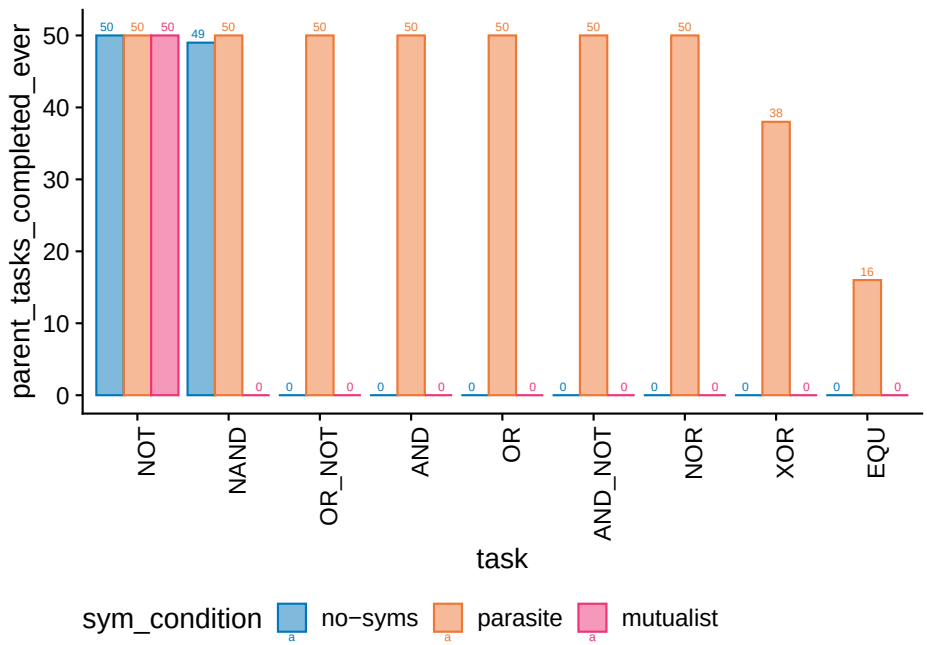
```
host_max_task_complexity_data <- host_over_time_parent_task_data %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier,
    task,
    task_complexity
  ) %>%
  summarize(
    n = n(),
    parent_task_completed_ever = any(task_completed_by_parent)
  ) %>%
  ungroup() %>%
  # This filter works fine because all replicates start with a task
  filter(
    parent_task_completed_ever
  ) %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier
  ) %>%
  summarize(
    max_task_complexity = max(task_complexity)
  ) %>%
  mutate(
    sym_condition = factor(
      sym_condition,
      levels = c(
```

```
        "no-syms",
        "parasite",
        "mutualist"
      )
    )
  )
```

5.4.1 Host function performance - all time

```
host_parent_tasks_all_time_by_task_plt <- host_parent_tasks_all_time_by_task %>%
  ggplot(
    aes(
      x = task,
      color = sym_condition,
      fill = sym_condition,
      y = parent_tasks_completed_ever
    )
  ) +
  geom_col(
    position = "dodge",
    alpha = 0.5
  ) +
  geom_text(
    aes(label = parent_tasks_completed_ever),
    position = position_dodge(width = 1),
    vjust = -0.5,
    size = 2
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  scale_x_discrete() +
  scale_y_continuous(
    limits = c(0, 51)
  ) +
  theme(
    legend.position = "bottom",
    axis.text.x = element_text(
      angle = 90,
      hjust = 1
    )
  )
ggsave(
  file = paste(
```

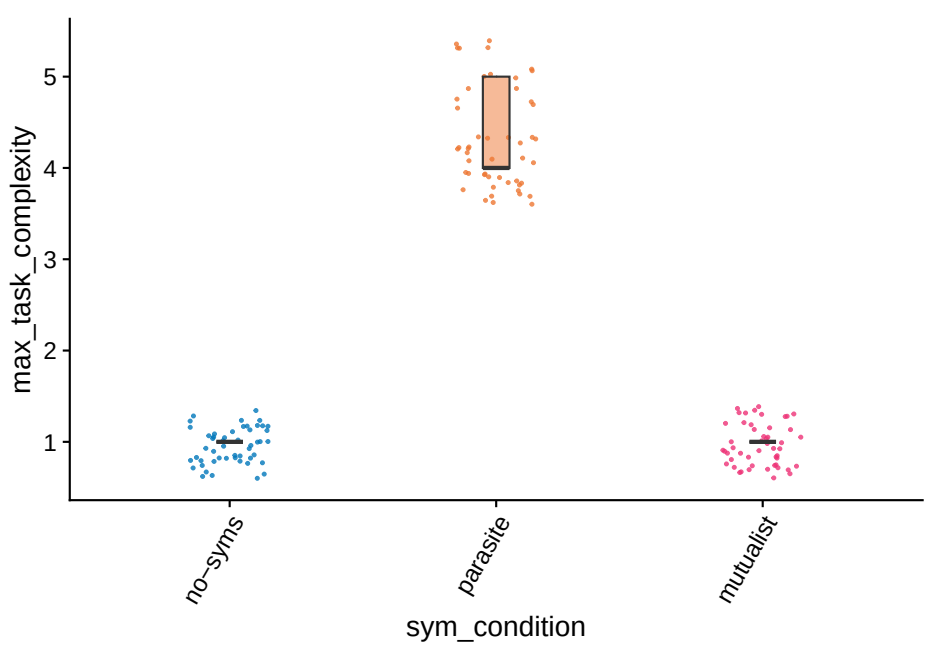
```
    plot_dir,
    "host_parent_tasks_all_time_by_task_plt.pdf",
    sep = "/"
  ),
  plot = host_parent_tasks_all_time_by_task_plt,
  width = 8,
  height = 8
)
host_parent_tasks_all_time_by_task_plt
```



5.4.2 Host maximum function complexity evolved

```
host_max_task_complexity_plt <- host_max_task_complexity_data %>%
  ggplot(
    aes(x = sym_condition, fill = sym_condition, y = max_task_complexity)
  ) +
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
```

```
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "host_max_task_complexity_plt.pdf", sep = "/"),
  plot = host_max_task_complexity_plt,
  width = 4,
  height = 4
)
host_max_task_complexity_plt
```



Kruskal-Wallis test to check significance among distribution differences.

```
kw_test <- host_max_task_complexity_data %>%
  group_by(interaction_multiplier) %>%
  rstatix::kruskal_test(max_task_complexity ~ sym_condition)
kw_test
```

```
## # A tibble: 1 x 7
##   interaction_multiplier .y.                   n statistic    df        p method
## * <fct>                  <chr>             <int>     <dbl> <int>    <dbl> <chr>
## 1 4                      max_task_complex~   150      144.     2 5.99e-32 Krusk~
```

Kruskal-Wallis test is significant, so we run Bonferroni-corrected pairwise Wilcoxon rank-sum tests to determine significant differences between pairs of treatments.

```
pairwise.wilcox.test(
  x = host_max_task_complexity_data$max_task_complexity,
  g = host_max_task_complexity_data$sym_condition,
  p.adjust.method = "bonferroni",
  exact = FALSE
)
```

```
##
##  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
##
## data:  host_max_task_complexity_data$max_task_complexity and host_max_task_complexi
##
##           no-syms parasite
## parasite  <2e-16  -
## mutualist -       <2e-16
##
## P value adjustment method: bonferroni
```

5.5 Symbiont function performance

```
# Pivot task data long, selecting only relevant columns
sym_over_time_parent_task_data <- time_data %>%
  select(
    SEED,
    sym_condition,
    interaction_multiplier,
    update,
```

```
    CurUpdate_AND_NOT_in_sym_parent_org_counts,
    CurUpdate_AND_in_sym_parent_org_counts,
    CurUpdate_EQU_in_sym_parent_org_counts,
    CurUpdate_NAND_in_sym_parent_org_counts,
    CurUpdate_NOR_in_sym_parent_org_counts,
    CurUpdate_NOT_in_sym_parent_org_counts,
    CurUpdate_OR_NOT_in_sym_parent_org_counts,
    CurUpdate_OR_in_sym_parent_org_counts,
    CurUpdate_XOR_in_sym_parent_org_counts
  ) %>%
  pivot_longer(
    c(
      CurUpdate_AND_NOT_in_sym_parent_org_counts,
      CurUpdate_AND_in_sym_parent_org_counts,
      CurUpdate_EQU_in_sym_parent_org_counts,
      CurUpdate_NAND_in_sym_parent_org_counts,
      CurUpdate_NOR_in_sym_parent_org_counts,
      CurUpdate_NOT_in_sym_parent_org_counts,
      CurUpdate_OR_NOT_in_sym_parent_org_counts,
      CurUpdate_OR_in_sym_parent_org_counts,
      CurUpdate_XOR_in_sym_parent_org_counts
    ),
    names_to = "task",
    values_to = "task_in_parent_count"
  ) %>%
  mutate(
    task = str_replace(task, "CurUpdate_", "")
  ) %>%
  mutate(
    task = str_replace(task, "_in_sym_parent_org_counts", "")
  ) %>%
  mutate(
    task_completed_by_parent = task_in_parent_count >= threshold,
    task = factor(
      task,
      levels = tasks,
      labels = tasks
    )
  ) %>%
  mutate(
    task_complexity = case_when(
      task == "NOT" ~ 1,
      task == "NAND" ~ 1,
      task == "OR_NOT" ~ 2,
      task == "AND" ~ 2,
```

```
      task == "OR" ~ 3,
      task == "AND_NOT" ~ 3,
      task == "NOR" ~ 4,
      task == "XOR" ~ 4,
      task == "EQU" ~ 5,
      .default = -1
    )
  )

sym_parent_tasks_all_time_by_task <- sym_over_time_parent_task_data %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier,
    task
  ) %>%
  summarize(
    n = n(),
    parent_task_completed_ever = any(task_completed_by_parent)
  ) %>%
  ungroup() %>%
  group_by(
    sym_condition,
    interaction_multiplier,
    task
  ) %>%
  summarize(
    n = n(),
    parent_tasks_completed_ever = sum(parent_task_completed_ever == TRUE)
  )
```

Next, summarize function complexity level.

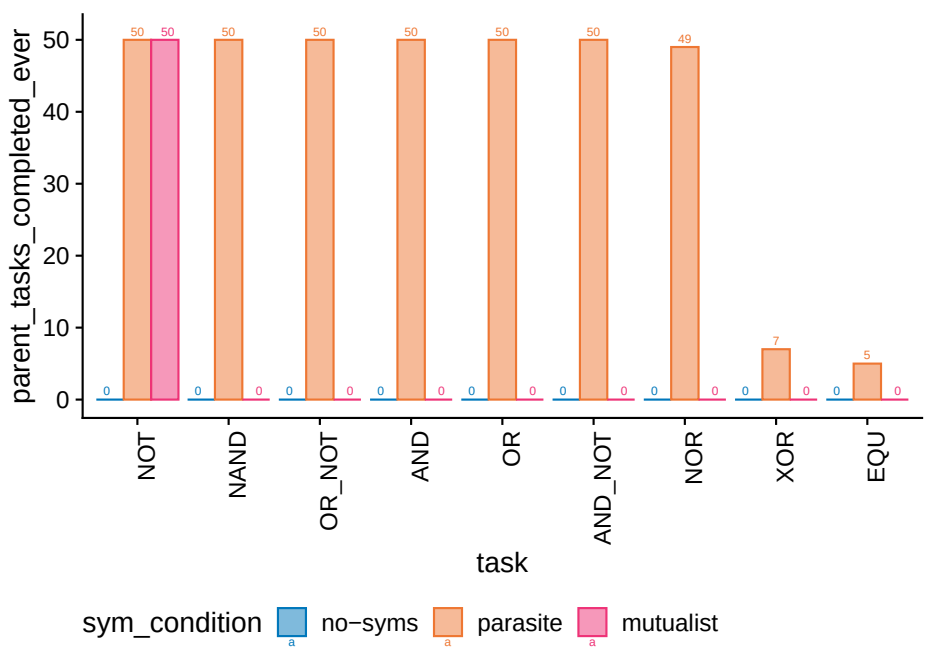
```
sym_max_task_complexity_data <- sym_over_time_parent_task_data %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier,
    task,
    task_complexity
  ) %>%
  summarize(
    n = n(),
    parent_task_completed_ever = any(task_completed_by_parent)
  ) %>%
```

```
  ungroup() %>%
  # This filter works fine because all replicates start with a task
  filter(
    parent_task_completed_ever
  ) %>%
  group_by(
    SEED,
    sym_condition,
    interaction_multiplier
  ) %>%
  summarize(
    max_task_complexity = max(task_complexity)
  ) %>%
  mutate(
    sym_condition = factor(
      sym_condition,
      levels = c(
        "no-syms",
        "parasite",
        "mutualist"
      )
    )
  )
```

5.5.1 Symbiont function performance - all time

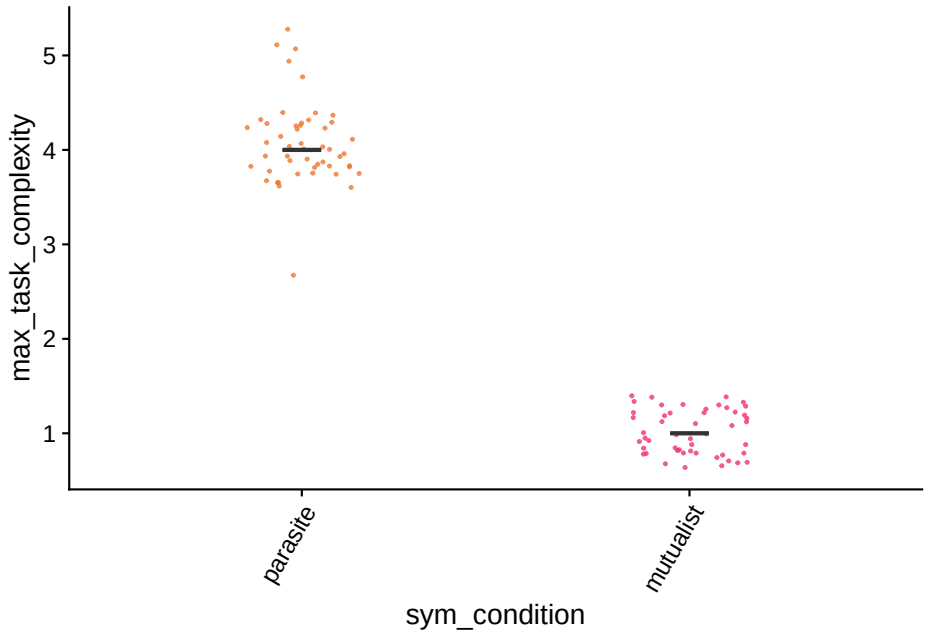
```
sym_parent_tasks_all_time_by_task_plt <- sym_parent_tasks_all_time_by_task %>%
  ggplot(
    aes(
      x = task,
      color = sym_condition,
      fill = sym_condition,
      y = parent_tasks_completed_ever
    )
  ) +
  geom_col(
    position = "dodge",
    alpha = 0.5
  ) +
  geom_text(
    aes(label = parent_tasks_completed_ever),
    position = position_dodge(width = 1),
    vjust = -0.5,
```

```
    size = 2
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  scale_x_discrete() +
  scale_y_continuous(
    limits = c(0, 51)
  ) +
  theme(
    legend.position = "bottom",
    axis.text.x = element_text(
      angle = 90,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "sym_parent_tasks_all_time_by_task_plt.pdf", sep = "/"),
  plot = sym_parent_tasks_all_time_by_task_plt,
  width = 8,
  height = 8
)
sym_parent_tasks_all_time_by_task_plt
```



5.5.2 Symbiont maximum function complexity evolved

```
sym_max_task_complexity_plt <- sym_max_task_complexity_data %>%
  ggplot(
    aes(x = sym_condition, fill = sym_condition, y = max_task_complexity)
  ) +
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  scale_color_manual(values = c(color_pal[2], color_pal[3])) +
  scale_fill_manual(values = c(color_pal[2], color_pal[3])) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "sym_max_task_complexity_plt.pdf", sep = "/"),
  plot = sym_max_task_complexity_plt,
  width = 4,
  height = 4
)
sym_max_task_complexity_plt
```



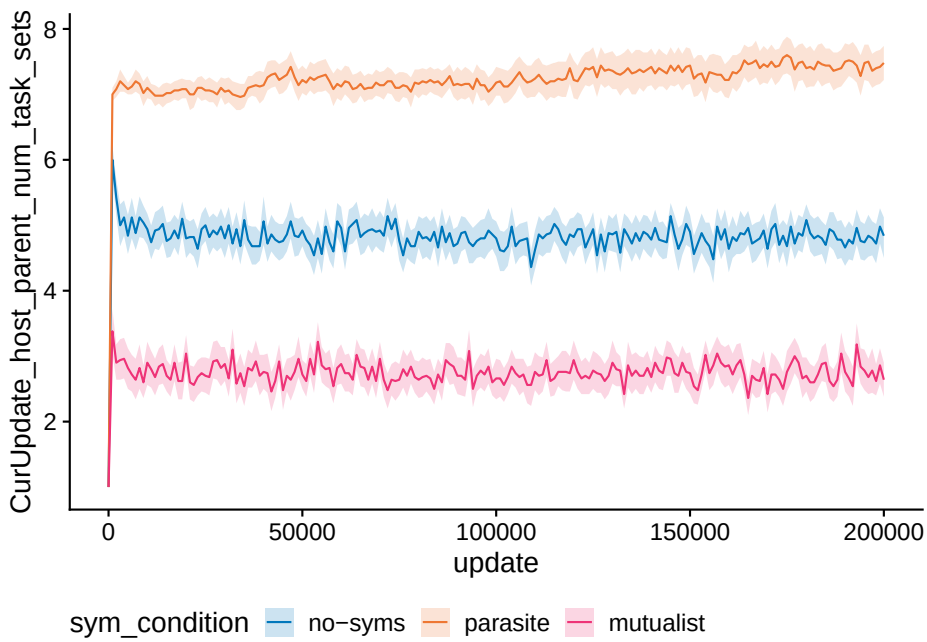
5.6 Host phenotypic diversity

5.6.1 Number of distinct function profiles

Over time:

```
host_parent_num_task_sets_plt <- time_data %>%
  # sample to make plotting faster
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = CurUpdate_host_parent_num_task_sets,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
```

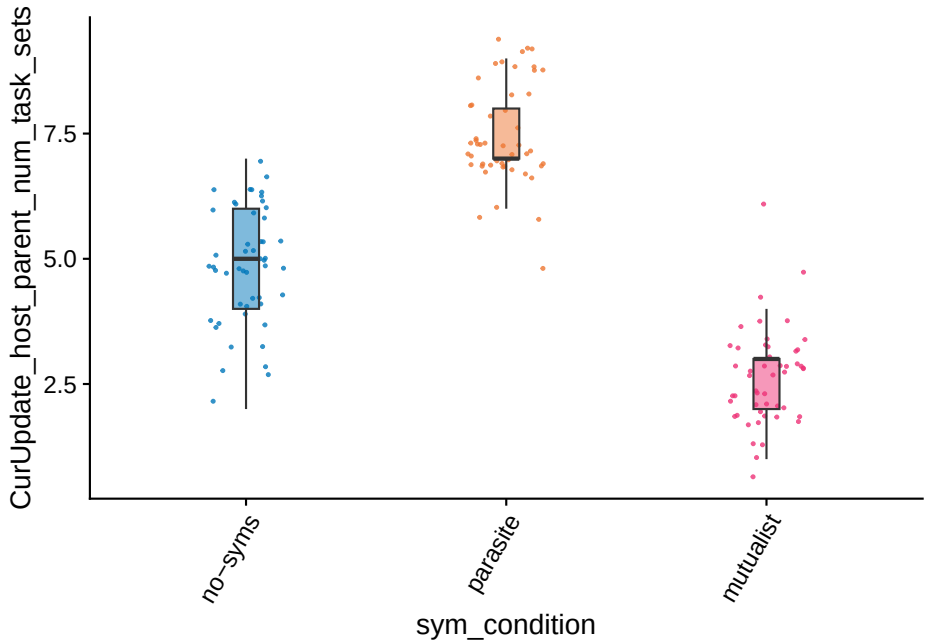
```
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(plot_dir, "host_parent_num_task_sets_plt.pdf", sep = "/"),
  plot = host_parent_num_task_sets_plt,
  width = 8,
  height = 8
)
host_parent_num_task_sets_plt
```



At the end of the experiment:

```
host_final_num_task_sets_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_host_parent_num_task_sets
    )
  ) +
```

```
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "host_final_num_task_sets_elapsed_plt.pdf", sep = "/"),
  plot = host_final_num_task_sets_elapsed_plt,
  width = 4,
  height = 4
)
host_final_num_task_sets_elapsed_plt
```

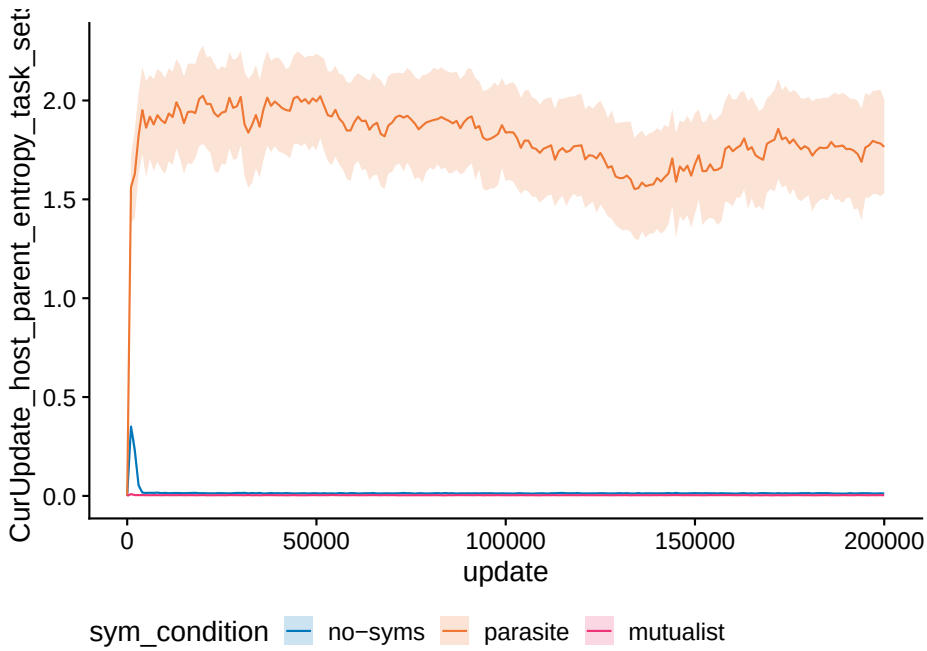


5.6.2 Shannon diversity of function profiles

Over time:

```
host_parent_entropy_task_sets_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = CurUpdate_host_parent_entropy_task_sets,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
```

```
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(plot_dir, "host_parent_entropy_task_sets_plt.pdf", sep = "/"),
  plot = host_parent_entropy_task_sets_plt,
  width = 8,
  height = 8
)
host_parent_entropy_task_sets_plt
```



At the end of the experiment:

```
host_final_entopy_task_sets_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_host_parent_entropy_task_sets
    )
  ) +
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
```

```
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "host_final_entopy_task_sets_elapsed_plt.pdf", sep = "/"),
  plot = host_final_entopy_task_sets_elapsed_plt,
  width = 10,
  height = 10
)
```

Statistical analysis

```
kw_test <- summary_data %>%
  group_by(interaction_multiplier) %>%
  rstatix::kruskal_test(CurUpdate_host_parent_entropy_task_sets ~ sym_condition)
kw_test
```

```
## # A tibble: 1 x 7
##   interaction_multiplier .y.                    n statistic    df        p method
## * <fct>                  <chr>              <int>     <dbl> <int>    <dbl> <chr>
## 1 4                      CurUpdate_host_p~   150      128.     2 1.38e-28 Krusk~
```

```
pairwise.wilcox.test(
  x = summary_data$CurUpdate_host_parent_entropy_task_sets,
  g = summary_data$sym_condition,
  p.adjust.method = "bonferroni",
  exact = FALSE
)
```

```
##
##  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
```

```
##
## data:  summary_data$CurUpdate_host_parent_entropy_task_sets and summary_data$sym_co
##
##           no-syms parasite
## parasite  < 2e-16 -
## mutualist 2.5e-15 < 2e-16
##
## P value adjustment method: bonferroni
```

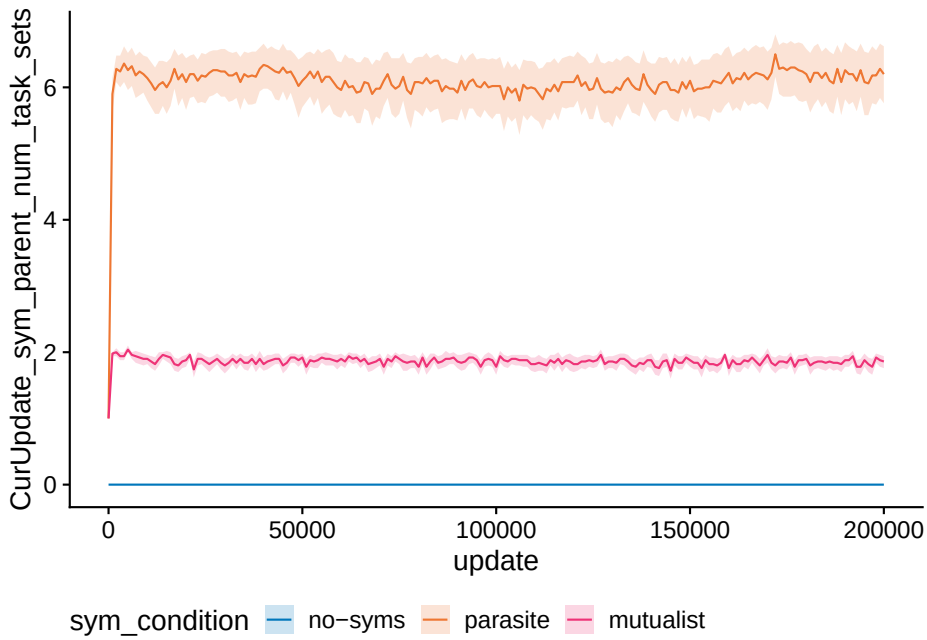
5.7 Symbiont phenotypic diversity

5.7.1 Number of distinct function profiles

Over time:

```
sym_parent_num_task_sets_plt <- time_data %>%
  # sample to make plotting faster
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = CurUpdate_sym_parent_num_task_sets,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(plot_dir, "sym_parent_num_task_sets_plt.pdf", sep = "/"),
  plot = sym_parent_num_task_sets_plt,
  width = 8,
  height = 8
```

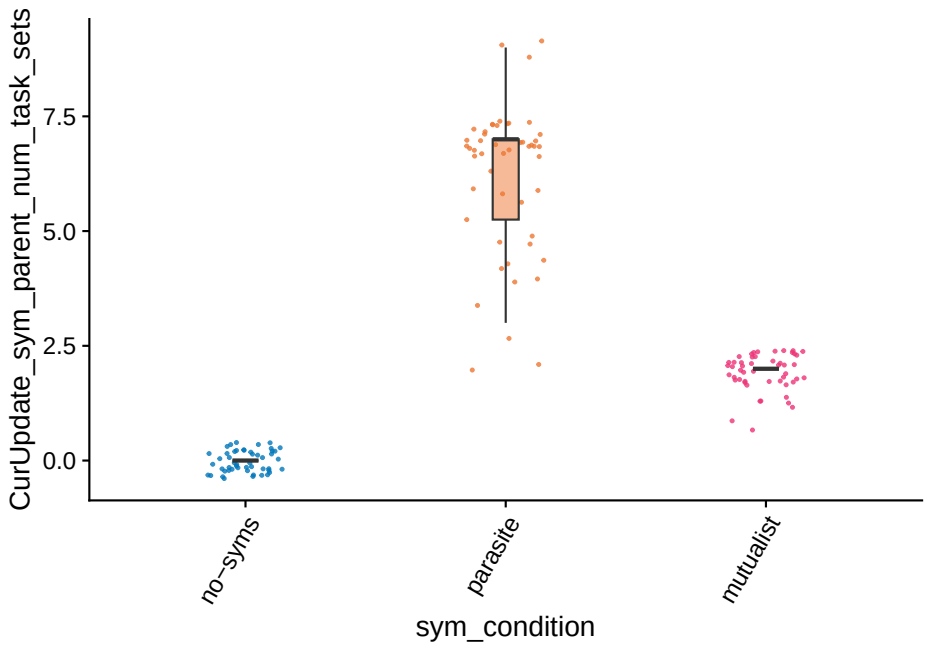
```
)
sym_parent_num_task_sets_plt
```



At the end of the experiment:

```
sym_final_num_task_sets_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_sym_parent_num_task_sets
    )
  ) +
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
```

```
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "sym_final_num_task_sets_elapsed_plt.pdf", sep = "/"),
  plot = sym_final_num_task_sets_elapsed_plt,
  width = 4,
  height = 4
)
sym_final_num_task_sets_elapsed_plt
```

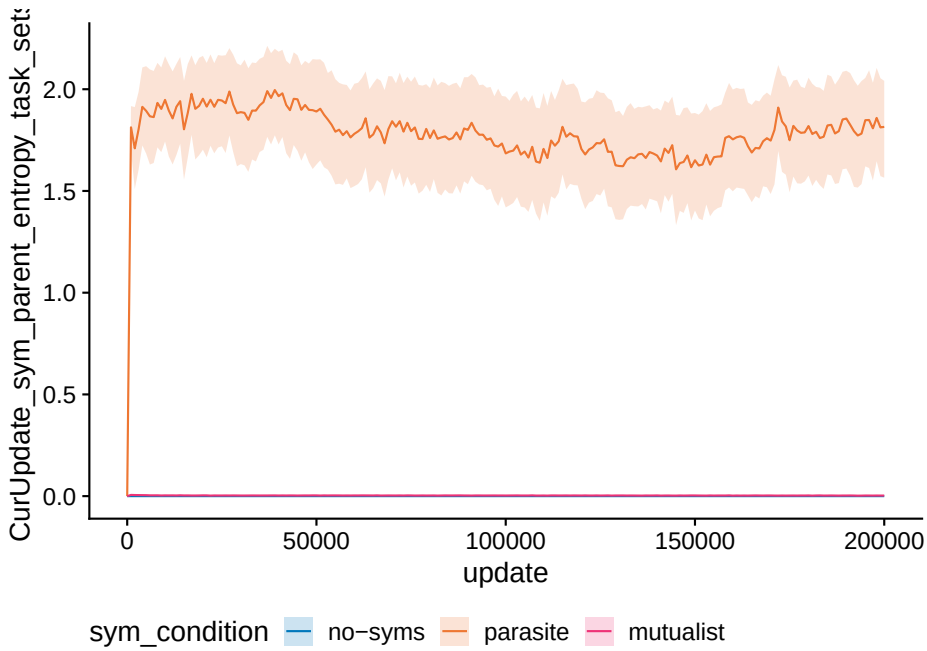


5.7.2 Shannon diversity of function profiles

Over time:

```
sym_parent_entropy_task_sets_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
```

```
  ggplot(
    aes(
      x = update,
      y = CurUpdate_sym_parent_entropy_task_sets,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(plot_dir, "sym_parent_entropy_task_sets_plt.pdf", sep = "/"),
  plot = sym_parent_entropy_task_sets_plt,
  width = 8,
  height = 8
)
sym_parent_entropy_task_sets_plt
```



At the end of the experiment:

```
sym_final_entopy_task_sets_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_sym_parent_entropy_task_sets
    )
  ) +
  geom_point(
    mapping=aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
```

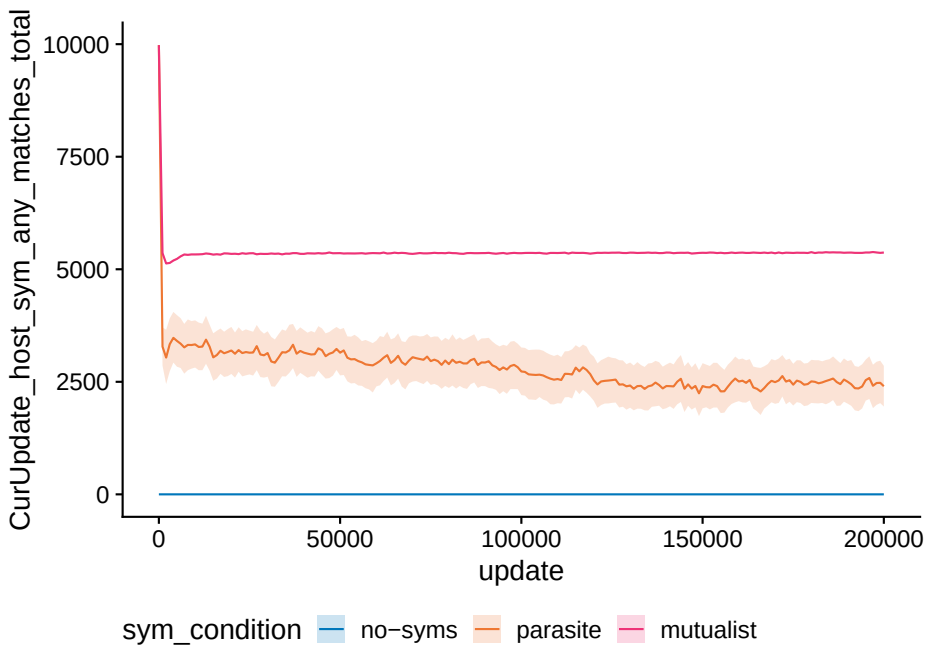
```
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "sym_final_entopy_task_sets_elapsed_plt.pdf", sep = "/"),
  plot = sym_final_entopy_task_sets_elapsed_plt,
  width = 10,
  height = 10
)
```

5.8 Trait matches over time

Counts of host-symbiont pairs with any trait matches. Note that organisms that were just born will not have had time to express any Boolean logic functions, so they will appear as not matching.

```
host_sym_any_matches_total_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = CurUpdate_host_sym_any_matches_total,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(plot_dir, "host_sym_any_matches_total_plt.pdf", sep = "/"),
  plot = host_sym_any_matches_total_plt,
```

```
  width = 8,
  height = 8
)
host_sym_any_matches_total_plt
```



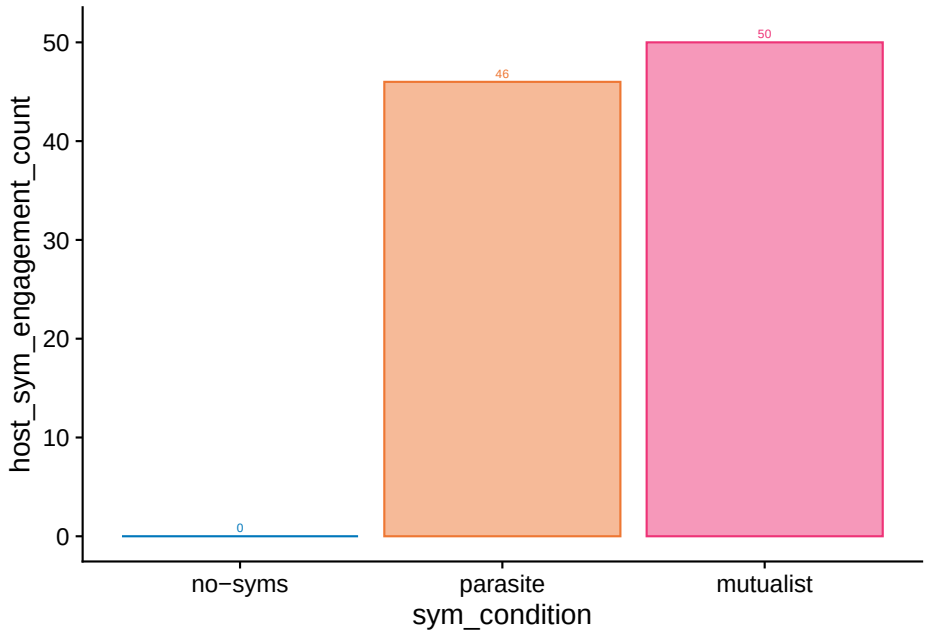
5.9 Evolutionary engagement

Use trait matching metrics to determine if host-symbiont populations are evolutionarily engaged.

```
engagement <- summary_data %>%
  mutate(
    host_sym_engagement = CurUpdate_host_sym_any_matches_total > threshold,
  ) %>%
  group_by(
    sym_condition
  ) %>%
  summarise(
    host_sym_engagement_count = sum(host_sym_engagement)
  )
```

```
host_sym_engagement_count_plt <- engagement %>%
  ggplot(
    aes(
      x = sym_condition,
      color = sym_condition,
      fill = sym_condition,
      y = host_sym_engagement_count
    )
  ) +
  geom_col(
    position = "dodge",
    alpha = 0.5
  ) +
  geom_text(
    aes(label = host_sym_engagement_count),
    position = position_dodge(width = 1),
    vjust = -0.5,
    size = 2
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  scale_x_discrete() +
  scale_y_continuous(
    limits = c(0, 51)
  ) +
  theme(
    legend.position = "none"
  )

ggsave(
  file = paste(plot_dir, "host_sym_engagement_count_plt.pdf", sep = "/"),
  plot = host_sym_engagement_count_plt,
  width = 4,
  height = 4
)
host_sym_engagement_count_plt
```

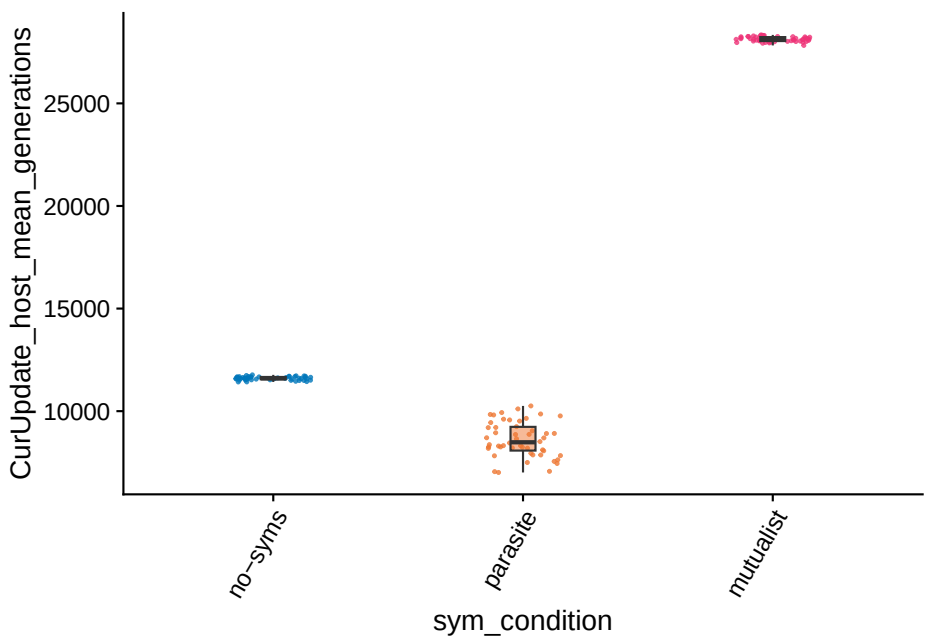


5.10 Generations of evolution elapsed

5.10.1 Hosts

```
host_avg_gens_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_host_mean_generations
    )
  ) +
  geom_point(
    mapping = aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
```

```
  ) +
  scale_x_discrete() +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "host_avg_gens_elapsed_plt.pdf", sep = "/"),
  plot = host_avg_gens_elapsed_plt,
  width = 4,
  height = 4
)
host_avg_gens_elapsed_plt
```




```
summary_data %>%
  group_by(
    sym_condition
  ) %>%
  summarise(
```

```
    mean_gens = mean(CurUpdate_host_mean_generations),
    median_gens = median(CurUpdate_host_mean_generations),
    var_gens = var(CurUpdate_host_mean_generations),
    sdev_gens = sd(CurUpdate_host_mean_generations)
  )
```

```
## # A tibble: 3 x 5
##   sym_condition mean_gens median_gens var_gens sdev_gens
##   <fct>             <dbl>       <dbl>    <dbl>     <dbl>
## 1 no-syms          11600.      11600     6960.      83.4
## 2 parasite          8621.       8484.  722611.     850.
## 3 mutualist        28125.      28122.   14533.     121.
```

```
kw_test <- summary_data %>%
  rstatix::kruskal_test(CurUpdate_host_mean_generations ~ sym_condition)
kw_test
```

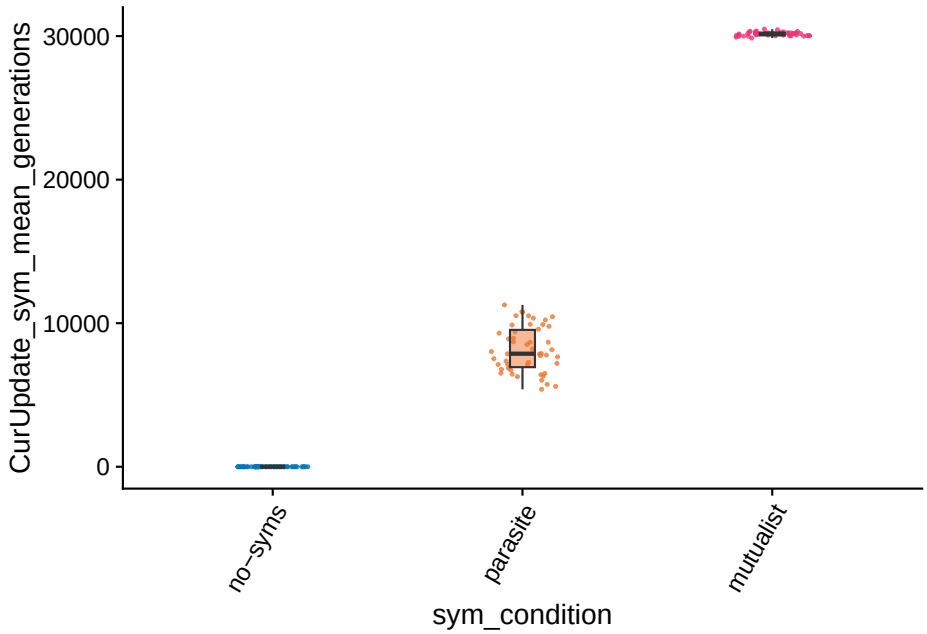
```
## # A tibble: 1 x 6
##   .y.                                n statistic    df        p method
## * <chr>                            <int>     <dbl> <int>    <dbl> <chr>
## 1 CurUpdate_host_mean_generations   150      132.     2 1.73e-29 Kruskal-Wallis
```

```
pairwise.wilcox.test(
  x = summary_data$CurUpdate_host_mean_generations,
  g = summary_data$sym_condition,
  p.adjust.method = "bonferroni",
  exact = FALSE
)
```

```
##
##  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
##
## data:  summary_data$CurUpdate_host_mean_generations and summary_data$sym_condition
##
##           no-syms parasite
## parasite  <2e-16  -
## mutualist <2e-16  <2e-16
##
## P value adjustment method: bonferroni
```

5.10.2 Symbionts

```
sym_avg_gens_elapsed_plt <- summary_data %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = CurUpdate_sym_mean_generations
    )
  ) +
  geom_point(
    mapping = aes(color = sym_condition),
    position = position_jitter(width = .15),
    size = .5,
    alpha = 0.8
  ) +
  geom_boxplot(
    width = .1,
    outlier.shape = NA,
    alpha = 0.5
  ) +
  scale_x_discrete() +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "none",
    axis.text.x = element_text(
      angle = 60,
      hjust = 1
    )
  )
ggsave(
  file = paste(plot_dir, "sym_avg_gens_elapsed_plt.pdf", sep = "/"),
  plot = sym_avg_gens_elapsed_plt,
  width = 4,
  height = 4
)
sym_avg_gens_elapsed_plt
```



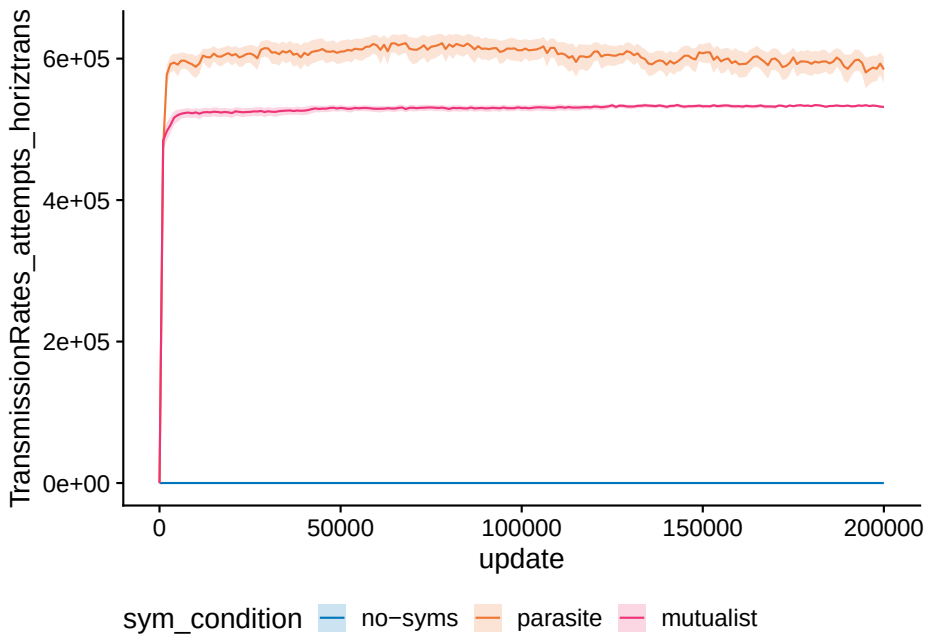

```
summary_data %>%
  group_by(
    sym_condition
  ) %>%
  summarise(
    mean_gens = mean(CurUpdate_sym_mean_generations),
    median_gens = median(CurUpdate_sym_mean_generations),
    var_gens = var(CurUpdate_sym_mean_generations),
    sdev_gens = sd(CurUpdate_sym_mean_generations)
  )
```

```
## # A tibble: 3 x 5
##   sym_condition mean_gens median_gens var_gens sdev_gens
##   <fct>             <dbl>       <dbl>    <dbl>     <dbl>
## 1 no-syms               0           0        0         0
## 2 parasite           8173.       7874. 2510452.     1584.
## 3 mutualist         30153.      30164.   16103.      127.
```

5.11 Symbiont transmission rates

5.11.1 Horizontal transmission attempts

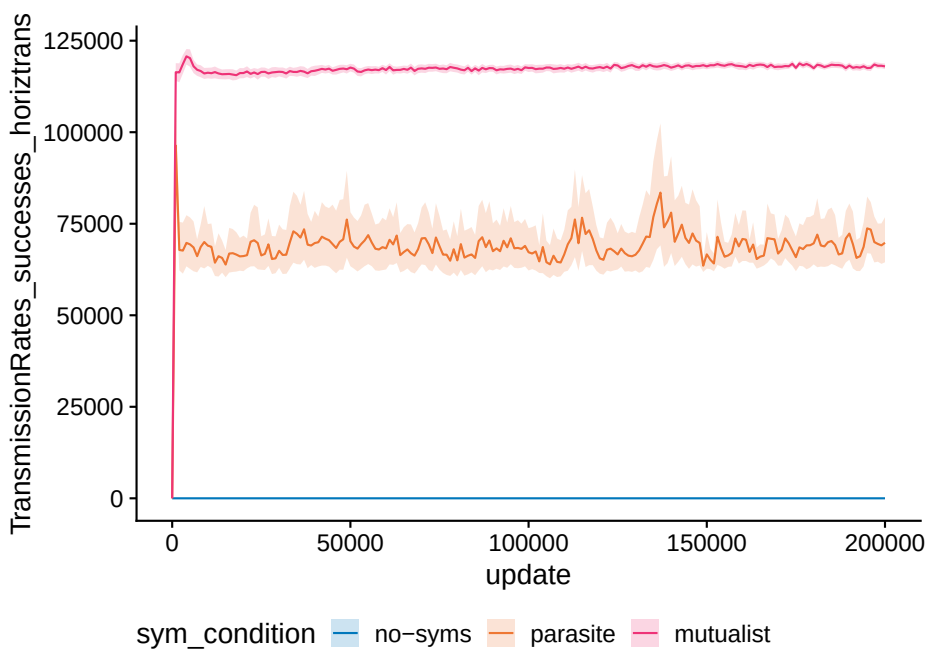
```
TransmissionRates_attempts_horiztrans_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = TransmissionRates_attempts_horiztrans,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "TransmissionRates_attempts_horiztrans_plt.pdf",
    sep = "/"
  ),
  plot = TransmissionRates_attempts_horiztrans_plt,
  width = 8,
  height = 8
)
TransmissionRates_attempts_horiztrans_plt
```



5.11.2 Horizontal transmission successes

```
TransmissionRates_successes_horiztrans_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = TransmissionRates_successes_horiztrans,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
```

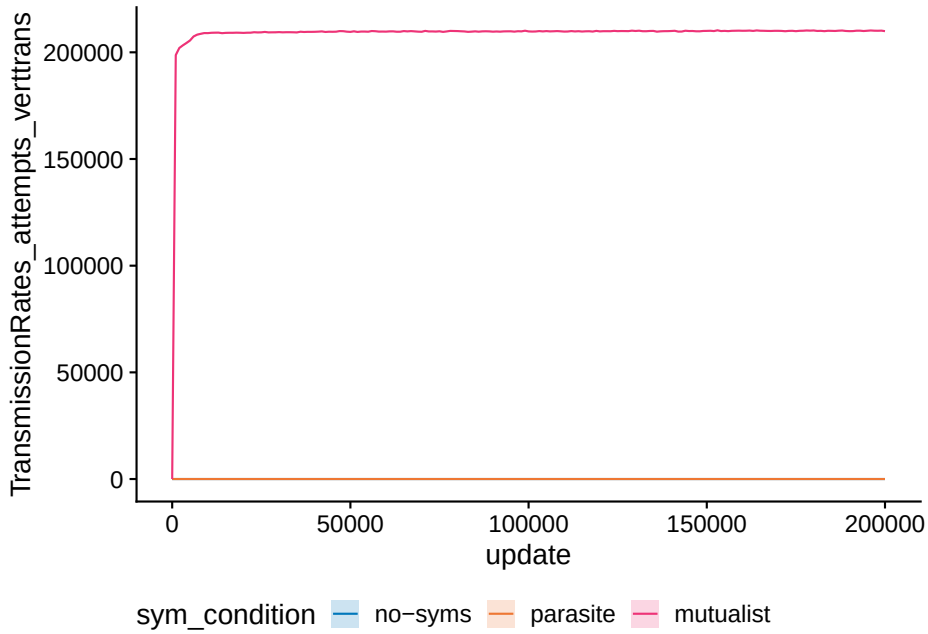
```
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "TransmissionRates_successes_horiztrans_plt.pdf",
    sep = "/"
  ),
  plot = TransmissionRates_successes_horiztrans_plt,
  width = 8,
  height = 8
)
TransmissionRates_successes_horiztrans_plt
```



5.11.3 Vertical transmission attempts

```
TransmissionRates_attempts_verttrans_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = TransmissionRates_attempts_verttrans,
```

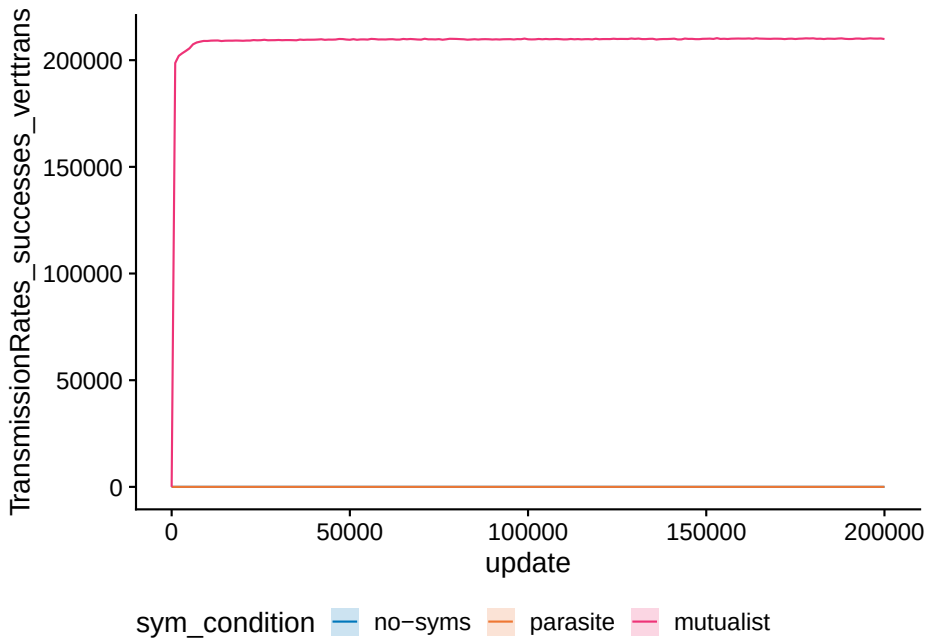
```
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "TransmissionRates_attempts_verttrans_plt.pdf",
    sep = "/"
  ),
  plot = TransmissionRates_attempts_verttrans_plt,
  width = 8,
  height = 8
)
TransmissionRates_attempts_verttrans_plt
```



5.11.4 Successes vertical transmission

```
TransmissionRates_successes_verttrans_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = TransmissionRates_successes_verttrans,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
```

```
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "TransmissionRates_successes_verttrans_plt.pdf",
    sep = "/"
  ),
  plot = TransmissionRates_successes_verttrans_plt,
  width = 8,
  height = 8
)
TransmissionRates_successes_verttrans_plt
```



5.12 Population size

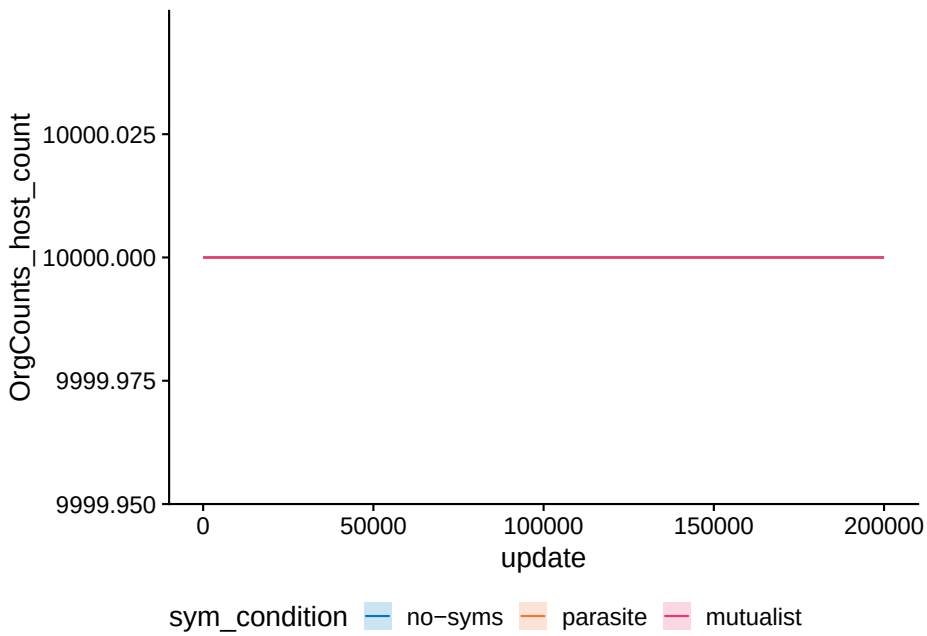
```
pop_size_summary <- time_data %>%
  group_by(
    SEED,
    sym_condition
  ) %>%
```

```
  summarize(
    host_pop_size_avg = mean(OrgCounts_host_count),
    host_pop_size_median = median(OrgCounts_host_count),
    host_pop_size_variance = var(OrgCounts_host_count),
    sym_pop_size_avg = mean(OrgCounts_hosted_sym_count),
    sym_pop_size_median = median(OrgCounts_hosted_sym_count),
    sym_pop_size_variance = var(OrgCounts_hosted_sym_count)
  )
```

5.12.1 Host population size

```
host_pop_size_ot_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = OrgCounts_host_count,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "host_pop_size_ot_plt.pdf",
    sep = "/"
  ),
  plot = host_pop_size_ot_plt,
  width = 8,
  height = 8
```

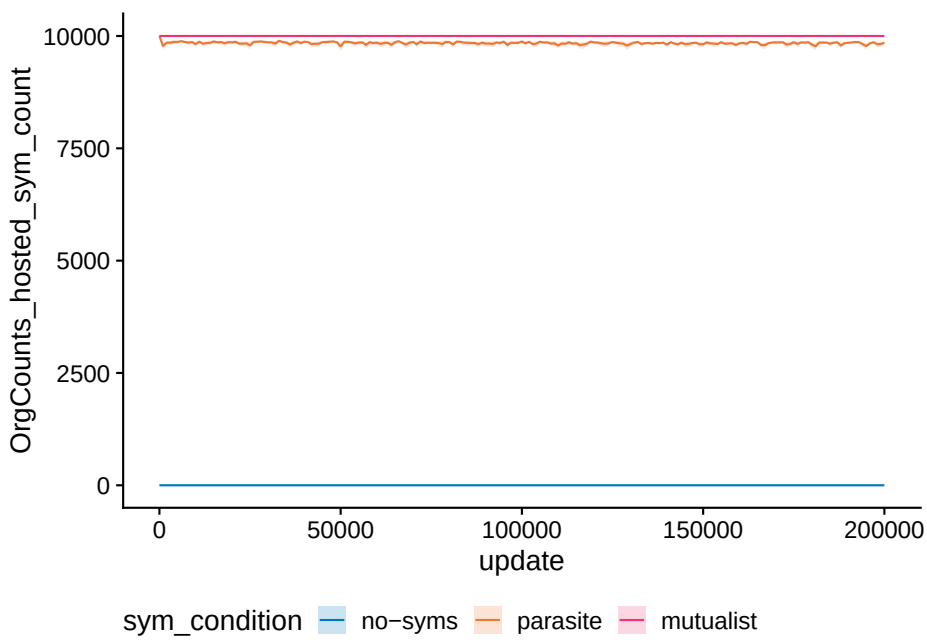
```
)
host_pop_size_ot_plt
```



5.12.2 Symbiont population size

```
sym_pop_size_ot_plt <- time_data %>%
  filter(update %% 1000 == 0) %>%
  ggplot(
    aes(
      x = update,
      y = OrgCounts_hosted_sym_count,
      color = sym_condition,
      fill = sym_condition
    )
  ) +
  stat_summary(fun = "mean", geom = "line") +
  stat_summary(
    fun.data = "mean_cl_boot",
    fun.args = list(conf.int = 0.95),
    geom = "ribbon",
    alpha = 0.2,
    linetype = 0
```

```
  ) +
  scale_color_manual(values = color_pal) +
  scale_fill_manual(values = color_pal) +
  theme(
    legend.position = "bottom"
  )
ggsave(
  file = paste(
    plot_dir,
    "sym_pop_size_ot_plt.pdf",
    sep = "/"
  ),
  plot = sym_pop_size_ot_plt,
  width = 8,
  height = 8
)
sym_pop_size_ot_plt
```



5.13 Manuscript figures

```
host_max_task_complexity_data_ci <- host_max_task_complexity_data %>%
  group_by(
```

```
    sym_condition
  ) %>%
  summarize(
    mean_max_task_complexity = mean(max_task_complexity),
    min_max_task_complexity = min(max_task_complexity),
    max_max_task_complexity = max(max_task_complexity),
    ci = list(
      mean_cl_boot(
        max_task_complexity,
        conf.int = 0.99,
        B = 10000,
      ) %>% rename(mean = y, lwr = ymin, upr = ymax)
    )
  ) %>%
  unnest(cols = c(ci)) %>%
  mutate(
    sym_condition = factor(
      sym_condition,
      levels = c("no-syms", "parasite", "mutualist"),
      labels = c("No symbionts", "Parasites", "Mutualists")
    )
  )

dist_data <- host_max_task_complexity_data %>%
  mutate(
    sym_condition = factor(
      sym_condition,
      levels = c("no-syms", "parasite", "mutualist"),
      labels = c("No symbionts", "Parasites", "Mutualists")
    )
  )

manuscript_host_task_complexity_plt <- host_max_task_complexity_data_ci %>%
  ggplot(
    aes(
      x = sym_condition,
      fill = sym_condition,
      y = mean_max_task_complexity
    )
  ) +
  geom_bar(stat = "identity") +
  geom_point(
    data = dist_data,
    mapping = aes(x = sym_condition, y = max_task_complexity),
    position = position_jitter(width = .4, height = 0.05),
```

```
    size = .7,
    alpha = 0.9,
    color = "darkgray",
    fill = "darkgray"
  ) +
  geom_linerange(
    aes(
      ymin = lwr,
      ymax = upr
    ),
    alpha = 0.9,
    linewidth = 0.7,
    color = "black"
  ) +
  scale_x_discrete(
    limits = c("No symbionts", "Parasites", "Mutualists"),
    labels = c("None", "Par.", "Mut.")
  ) +
  scale_fill_manual(values = color_pal) +
  labs(
    x = "Sym. type",
    y = "Host function complexity"
  ) +
  theme(
    legend.position = "none",
    axis.title.y = element_text(size = 10),
    axis.text.y = element_text(size = 8),
    axis.title.x = element_text(size = 10),
    axis.text.x = element_text(size = 8)
  )


ggsave(
  file = paste(
    plot_dir,
    "manuscript_flat_rewards_host_task_complexity_plt.pdf",
    sep = "/"
  ),
  plot = manuscript_host_task_complexity_plt,
  width = 3,
  height = 3
)
```